

Smart Document Analyst

Building a Multi-Agent AI System for Scanned Document Analysis

AI & Big Data Program – UIR

Team Members:

Zatri Abderahmane

Fazouane Oualid

Ibenouazi Ismail

Supervised by:

Prof. Youness Moukafih

Course Professor:

Prof. Hakim Hafidi

2025–2026

Abstract

This report presents the design, implementation, and evaluation of **Smart Document Analyst**, a multi-agent AI system for analyzing scanned document images. The system classifies document images using a trained PyTorch CNN model, extracts text using OCR, summarizes the extracted content, requires human approval, and generates a structured final report. The normal execution is handled by CrewAI agents using their assigned tools, with an Ollama LLM backend providing the reasoning layer. Guardrails and workflow state are used to reduce hallucinated outputs and ensure that final results come from real tool executions. A deterministic Python workflow remains available only for no-LLM execution or optional fallback. The project follows the Smart Document Analyst domain: classify documents, extract and summarize content, and generate reports.

Contents

1	Introduction	3
2	Problem Definition	3
3	System Scope	3
3.1	Supported Inputs	3
3.2	Supported Classes	4
3.3	Out of Scope	4
4	Technical Stack	4
5	Multi-Agent Architecture	5
5.1	Architecture Overview	5

5.2	Agent Roles	5
5.3	Design Rationale	6
6	Tools and Schemas	6
6.1	CNN Classifier Tool	7
6.2	Human-in-the-Loop Tool	7
7	Deep Learning Model	7
7.1	Model Choice	7
7.2	Model Input	8
7.3	Transfer Learning Details	8
7.4	Model Integration as Agent Tool	8
8	Model Evaluation	9
8.1	Evaluation Metrics	9
8.2	Evaluation Analysis	9
9	End-to-End Workflow	10
9.1	Execution Command	10
9.2	Example Result	10
10	Logging and Error Handling	10
10.1	JSON Logging	10
10.2	Error Handling	11
11	Robustness and Limitations	11
11.1	Robustness Features	11
11.2	Current Limitations	12
12	Reproducibility	12
13	Conclusion	13

1 Introduction

The objective of this project is to design, build, and evaluate a multi-agent AI system where specialized agents collaborate to solve a real-world problem. The selected domain is **Smart Document Analyst**. The system analyzes scanned document images and produces a structured report containing the predicted document type, confidence score, extracted text, key fields, summary, human approval status, and robustness notes.

The project focuses on scanned document images because the deep learning component is a Convolutional Neural Network (CNN). CNNs are naturally suited for visual classification tasks, especially when the document layout, typography, and visual structure are important. Therefore, the system supports image formats such as PNG, JPG, JPEG, TIFF, and BMP. PDF, TXT, DOCX, and other non-image formats are not supported in the current version.

The system satisfies the main requirements of the integrated project:

- a multi-agent architecture with specialized agents;
- at least two specialist agents and one orchestrator;
- a trained PyTorch deep learning model used as a functional tool;
- at least two tools with clear input and output schemas;
- human-in-the-loop approval before final report generation;
- JSON logging with timestamps;
- error handling and robust fallback execution;
- evaluation of the deep learning model using accuracy and confusion matrix.

2 Problem Definition

Organizations often receive large amounts of scanned documents, such as invoices, letters, memos, resumes, and reports. Manually identifying the type of each document and extracting useful information can be slow and repetitive. A smart document analysis system can reduce this manual effort by automatically classifying documents, extracting text, and producing a concise report.

The main problem addressed in this project is:

How can a multi-agent AI system classify scanned documents, extract useful information, summarize the content, and generate a reliable report while keeping a human approval checkpoint?

The system does not aim to be a universal document understanding platform. Instead, it focuses on a controlled and explainable workflow where every component has a clear role.

3 System Scope

3.1 Supported Inputs

The system supports scanned document images only:

- .png
- .jpg
- .jpeg
- .tif
- .tiff
- .bmp

This scope is intentional. Since the classifier is a CNN model, the system analyzes the visual appearance and layout of scanned documents.

3.2 Supported Classes

The trained classifier predicts one of five classes:

Class	Description
invoice	Commercial or billing document
letter	Formal or informal letter
memo	Short internal communication document
resume	Curriculum vitae or professional profile
scientific_report	Academic or scientific report

3.3 Out of Scope

The following features are outside the current project scope:

- PDF parsing;
- DOCX processing;
- handwritten document recognition;
- complex table extraction;
- multilingual OCR optimization;
- deployment as a web application.

4 Technical Stack

The project follows the required technical stack and uses the following technologies:

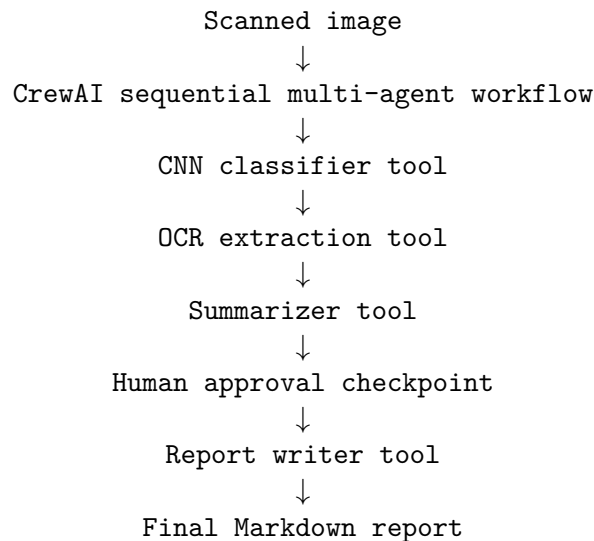
Component	Technology	Purpose
Agent framework	CrewAI	Multi-agent execution and orchestration
LLM backend	Ollama / Llama 3.1	Local LLM reasoning layer
Deep learning	PyTorch	CNN model training and inference
Model architecture	ResNet18	Transfer learning classifier
OCR	Tesseract OCR	Text extraction from images
Language	Python 3.10+	Main implementation language
Logging	JSONL	Timestamped agent/tool traces
Report format	Markdown	Generated final analysis reports

5 Multi-Agent Architecture

5.1 Architecture Overview

The system uses CrewAI as the multi-agent framework. In normal execution mode, the workflow is executed through CrewAI sequential tasks. Each specialist agent owns specific tools and is responsible for executing the step that matches its role. The LLM provides the reasoning layer for the agents, while tools perform the actual deterministic operations such as CNN inference, OCR extraction, summarization, approval handling, and report writing.

The workflow can be summarized as follows:



The execution order is controlled using CrewAI tasks with `Process.sequential`. This means that the workflow order is explicitly defined as classification, extraction, summarization, human approval, and final report generation. The LLM does not freely invent the global workflow order at runtime; instead, it reasons and acts within a controlled multi-agent structure.

5.2 Agent Roles

Agent	Role
Orchestrator Agent	Manager Writes the execution plan and ensures that the workflow follows the required order: classification, OCR extraction, summarization, human approval, and report generation.

Document Classification Agent	Classification	Uses the <code>cnn_document_classifier</code> tool to classify the scanned document image with the trained ResNet18 transfer-learning model.
Extraction and Summarization Agent	Summarization	Uses the <code>document_extraction_tool</code> to extract OCR text and the <code>local_summarizer_tool</code> to summarize the real extracted text.
Report Agent	Generation	Uses the <code>human_approval_checkpoint</code> tool before report generation, then uses the <code>structured_report_writer</code> tool to create the final Markdown report.

5.3 Design Rationale

The final design uses CrewAI sequential tasks with specialist agents and real tool execution. This makes the system easier to explain and more reliable than a fully open-ended autonomous workflow. Each agent has a restricted set of tools, and each task has a clear expected output.

During testing, LLM agents sometimes produced fake function-call JSON or described a tool call instead of executing the actual tool. To address this issue, the system includes guardrails and workflow state. Guardrails validate the outputs of each task and reject hallucinated or placeholder answers. Workflow state stores real outputs from the classifier, extractor, summarizer, approval checkpoint, and report writer so later tools can reuse verified results.

This design improves:

- reliability;
- reproducibility;
- error handling;
- explainability;
- protection against hallucinated file paths or fake outputs.

A deterministic Python workflow is still available, but it is not the default execution path. It is used only when the user explicitly runs the system with `-no-llm`, or when optional fallback is enabled with `-fallback-manual`.

6 Tools and Schemas

The project includes multiple tools. Each tool has a clear responsibility and a defined input/output behavior.

Tool	Function
<code>cnn_document_classifier</code>	Classifies the scanned document image using the trained PyTorch CNN model.
<code>document_extraction_tool</code>	Extracts OCR text and detects simple key fields such as dates, emails, amounts, or phone numbers.
<code>local_summarizer_tool</code>	Produces an extractive summary from OCR text.
<code>human_approval_checkpoint</code>	Requires user approval before the final report is generated.
<code>structured_report_writer</code>	Writes the final Markdown analysis report.

6.1 CNN Classifier Tool

The CNN classifier tool receives a scanned image path and returns:

- predicted class;
- confidence score;
- probability for each class;
- human review flag;
- model architecture.

Example output:

```
{
  "predicted_class": "resume",
  "confidence": 0.9997,
  "class_probabilities": {
    "invoice": 0.0000,
    "letter": 0.0001,
    "memo": 0.0000,
    "resume": 0.9997,
    "scientific_report": 0.0001
  },
  "needs_human_review": false,
  "model_architecture": "resnet18_transfer_learning"
}
```

6.2 Human-in-the-Loop Tool

The system includes a human approval checkpoint before final report generation. The user sees the predicted class, confidence score, and summary preview, then must approve the generation of the final report.

Example:

```
HUMAN-IN-THE-LOOP CHECKPOINT
Predicted class: resume (confidence=1.000)
Summary preview:
...
Approve final report generation? [y/N]: y
```

For demo and automated testing, the system also supports automatic approval with:

```
--auto-approve
```

7 Deep Learning Model

7.1 Model Choice

The deep learning component is a PyTorch CNN based on **ResNet18 transfer learning**. ResNet18 was selected because it is a proven CNN architecture for image classification tasks and works well with limited datasets when transfer learning is used.

The trained model is saved as:

```
models/document_cnn.pt
```

The current model checkpoint stores the architecture name as:

```
resnet18_transfer_learning
```

7.2 Model Input

Input images are converted to RGB, resized to 224×224 , transformed into tensors, and normalized using ImageNet-style normalization:

```
Resize((224, 224))
ToTensor()
Normalize(mean=[0.485, 0.456, 0.406],
          std=[0.229, 0.224, 0.225])
```

7.3 Transfer Learning Details

The model uses a ResNet18 architecture provided by TorchVision. The original ResNet18 feature extraction layers are reused from pretrained weights, and the final classification layer is replaced to match the number of project classes.

The training code follows this logic:

```
weights = models.ResNet18_Weights.DEFAULT
model = models.resnet18(weights=weights)
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

This means that the model is not trained from scratch. It starts from pretrained ResNet18 weights and adapts the final classification head to the project classes: `invoice`, `letter`, `memo`, `resume`, and `scientific_report`.

The project also contains a legacy custom CNN implementation in:

```
src/sda/models/cnn.py
```

However, the current trained model uses ResNet18 transfer learning. The custom CNN file remains only as a backward-compatible fallback for older checkpoints.

7.4 Model Integration as Agent Tool

The trained model is not decorative. It is wrapped inside the `CNNDocumentClassifierTool` and called during the CrewAI workflow by the Document Classification Agent. The classification output affects the report content and the human review logic. If the confidence is below the configured threshold, the output is marked as requiring human review.

This satisfies the requirement that at least one agent must use a deep learning model trained by the team as a functional tool.

8 Model Evaluation

The model was evaluated on a test set of 237 scanned document images. The evaluation outputs are stored in:

```
outputs/evaluation/
```

The evaluation includes:

- test accuracy;
- precision;
- recall;
- F1-score;
- support per class;
- confusion matrix.

8.1 Evaluation Metrics

The achieved test accuracy is:

81.01%

The detailed classification report is:

Class	Precision	Recall	F1-score	Support
invoice	0.87	0.85	0.86	46
letter	0.81	0.86	0.83	49
memo	0.70	0.79	0.75	48
resume	0.88	0.89	0.88	47
scientific_report	0.82	0.66	0.73	47
accuracy		0.81		237
macro avg	0.81	0.81	0.81	237
weighted avg	0.81	0.81	0.81	237

8.2 Evaluation Analysis

The model performs well overall, especially for resumes and invoices. The weakest class is `scientific_report`, with a recall of 0.66. This means that some scientific reports may be confused with visually similar classes. This limitation is important and should be considered during deployment.

The confusion matrix is saved as:

```
outputs/evaluation/confusion_matrix.png
```

If this image is included in the final report folder, it can be inserted using:

```
\includegraphics[width=0.75\textwidth]{outputs/evaluation/confusion_matrix.png}
```

9 End-to-End Workflow

9.1 Execution Command

The main command used for the final demo is:

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --auto-approve
```

This command runs the normal CrewAI workflow. The `-auto-approve` option is used only to avoid manual input during demo or testing.

Before running, the environment variables can be configured:

```
$env:OLLAMA_MODEL="ollama/llama3.1"
$env:OLLAMA_BASE_URL="http://localhost:11434"
$env:Path += ";C:\Program Files\Tesseract-OCR"
```

9.2 Example Result

For the sample resume image, the classifier produced:

- predicted class: `resume`;
- confidence: `0.9997`;
- model architecture: `resnet18_transfer_learning`.

The OCR extraction produced:

- word count: `151`;
- detected key field: `date: July 8, 1933`.

The final report was generated after approval and saved in:

```
outputs/reports/
```

10 Logging and Error Handling

10.1 JSON Logging

Every important action is logged in JSONL format with timestamps. The log file is:

```
outputs/logs/agent_actions.jsonl
```

Example log actions include:

- `build_crewai_agents`;
- `start_analysis`;

- `crewai_kickoff;`
- `agent_tool_call;`
- `cnn_classification_tool;`
- `extract_document_text;`
- `summarize_document;`
- `human_approval_checkpoint;`
- `write_final_report;`
- `finish_analysis.`

10.2 Error Handling

The system handles several error cases:

- missing document path;
- unsupported file extension;
- OCR failure when Tesseract is not installed;
- LLM backend failure;
- invalid tool outputs;
- hallucinated or fake function-call JSON;
- low confidence classification.

Guardrails validate the outputs of CrewAI tasks and reject placeholder or fake responses. Workflow state stores verified tool results so that later steps can reuse the real classifier, OCR, summarizer, approval, and report outputs.

A deterministic no-LLM workflow is available with:

```
--no-llm
```

Optional fallback can be enabled with:

```
--fallback-manual
```

In normal mode, the system runs through CrewAI agents and their assigned tools.

11 Robustness and Limitations

11.1 Robustness Features

The project includes the following robustness mechanisms:

- confidence threshold for human review;

- human-in-the-loop approval before report generation;
- auto-approval option for demo/testing;
- JSON logs for traceability;
- CrewAI sequential task execution;
- tool ownership by specialist agents;
- guardrails against fake outputs;
- workflow state for sharing verified results between tools;
- no-LLM mode for reproducibility;
- optional fallback mode for robustness.

11.2 Current Limitations

The current system has some limitations:

- only scanned images are supported;
- OCR quality depends on Tesseract and image quality;
- key-field extraction is rule-based and limited;
- scientific reports have lower recall than other classes;
- the workflow order is defined by CrewAI tasks rather than freely planned by the LLM at runtime;
- guardrails are needed because local LLM tool-calling can sometimes produce fake function-call text instead of executing tools.

These limitations are acceptable for the project scope because the goal is not to build the most complex system, but to build a system where every component exists for a reason and can be explained.

12 Reproducibility

The project includes a README file with setup instructions. To reproduce the final demo, a user must install:

- Python dependencies from `requirements.txt`;
- Tesseract OCR;
- Ollama;
- Llama 3.1 model through Ollama.

Then run the normal CrewAI workflow:

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --auto-approve
```

For manual human approval, remove `-auto-approve`:

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg"
```

For testing without LLM:

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --no-llm --auto-approve
```

For optional manual fallback if CrewAI fails:

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --fallback-manual --auto-approve
```

13 Conclusion

Smart Document Analyst successfully implements a multi-agent AI system for scanned document analysis. The system combines CrewAI and Ollama for multi-agent reasoning and orchestration, a trained PyTorch ResNet18 CNN for document classification, Tesseract OCR for text extraction, a local summarizer for content summarization, and a human-in-the-loop checkpoint for controlled report generation.

The project demonstrates that a deep learning model can be meaningfully integrated as a functional tool inside a larger AI system. In normal execution, the system runs primarily through CrewAI agents and their assigned tools. Guardrails, workflow state, and optional deterministic modes improve reliability, reduce hallucination risk, and make the outputs traceable through JSON logging.

The system satisfies the main project requirements: multi-agent architecture, trained DL model integration, working end-to-end demo, HITL checkpoint, error handling, logging, evaluation, and final report generation.

Future improvements could include PDF support, better OCR preprocessing, stronger key-field extraction, improved dataset size, and deployment through a simple web interface.

Appendix: Useful Commands

Run Final Demo

```
$env:OLLAMA_MODEL="ollama/llama3.1"
$env:OLLAMA_BASE_URL="http://localhost:11434"
$env:Path += ";C:\Program Files\Tesseract-OCR"

python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --auto-approve
```

Run With Manual Approval

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg"
```

View Logs

```
python main.py logs --tail 30
```

Run Without LLM

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --no-llm --auto-approve
```

Run With Optional Manual Fallback

```
python main.py analyze --document "data\real_raw\docs-sm\resume\00071736_00071737.jpg" --fallback-manual --auto-approve
```

Check Tesseract

```
tesseract --version  
python -c "import pytesseract; print(pytesseract.get_tesseract_version())"
```

Check Ollama

```
ollama --version  
ollama list
```

Train CNN Model

```
python scripts\train_cnn.py
```

Evaluate CNN Model

```
python scripts\evaluate_cnn.py
```